Geth 성능 모니터링과 튜닝 분석

개념적 배경

왜 성능 모니터링이 중요한가?

Ethereum 노드의 성능은 다양한 요인에 의해 영향을 받습니다:

1. **복잡한 시스템 상호작용**: 네트워킹, 스토리지, 컴퓨팅이 복합적으로 작용
2. **동적 워크로드**: 네트워크 활동에 따른 부하 변화
3. **리소스 제약**: 메모리, CPU, 디스크 I/O의 균형 조정 필요
4. **성능 병목 식별**: 실시간 모니터링을 통한 문제점 파악

Geth의 성능 모니터링 전략

Geth는 다층적 모니터링 시스템을 제공합니다:

```
Application Metrics
↓
System Metrics (CPU, Memory, I/O)
↓
Runtime Metrics (GC, Goroutines)
↓
Custom Metrics (Blockchain-specific)
```

구현 분석

1. 메트릭 수집 시스템 (`metrics/`)

메트릭 레지스트리

```
// 메트릭 레지스트리 인터페이스
type Registry interface {
Each(func(string, interface{})) // 모든 메트릭 순회
Get(string) interface{} // 특정 메트릭 조회
GetAll() map[string]map[string]interface{} // 모든 메트릭 데이터
GetOrRegister(string, func() interface{}) interface{} // 메트릭 등록/조회
Register(string, interface{}) error  // 메트릭 등록
RunHealthchecks() // 헬스체크 실행
Unregister(string) // 메트릭 해제
}

// 표준 레지스트리 구현
type StandardRegistry struct {
metrics sync.Map // 동시성 안전한 메트릭 저장소
}

func (r *StandardRegistry) GetOrRegister(name  string, ctor  func() interface{})
// 빠른 경로: 기존 메트릭 확인
```

```
cached, ok := r.metrics.Load(name)

if ok {

return cached

}

// 새 메트릭 생성 및 등록

item, _, _ := r.loadOrRegister(name, ctor())

return item

}
```

런타임 통계 수집

```
// 런타임 통계 구조체

type runtimeStats struct {

GCPauses *metrics.Float64Histogram // GC 일시정지 시간

GCAllocBytes uint64 // GC 할당 바이트

GCFreedBytes uint64 // GC 해제 바이트

MemTotal uint64 // 총 메모리

HeapObjects uint64 // 힙 객체 수

HeapFree uint64 // 힙 여유 공간

HeapReleased uint64 // 힙 해제 공간

HeapUnused uint64 // 힙 미사용 공간

Goroutines uint64 // 고루틴 수

SchedLatency *metrics.Float64Histogram // 스케줄링 지연시간
```

```
}

// 런타임 메트릭 샘플 정의
var runtimeSamples = []metrics.Sample{
{Name: "/gc/pauses:seconds"},
{Name: "/gc/heap/allocs:bytes"},
{Name: "/gc/heap/frees:bytes"},
{Name: "/memory/classes/total:bytes"},
{Name: "/memory/classes/heap/objects:bytes"},
{Name: "/memory/classes/heap/free:bytes"},
{Name: "/memory/classes/heap/released:bytes"},
{Name: "/memory/classes/heap/unused:bytes"},
{Name: "/sched/goroutines:goroutines"},
{Name: "/sched/latencies:seconds"},
}

func ReadRuntimeStats() *runtimeStats {
r := new(runtimeStats)
readRuntimeStats(r)
return r
}

func readRuntimeStats(v *runtimeStats) {
```

```
metrics.Read(runtimeSamples)

for _, s := range runtimeSamples {

if s.Value.Kind() == metrics.KindBad {

continue // 유효하지 않은 메트릭 스킵

}

switch s.Name {

case "/gc/pauses:seconds":

v.GCPauses = s.Value.Float64Histogram()

case "/gc/heap/allocs:bytes":

v.GCAllocBytes = s.Value.Uint64()

case "/gc/heap/frees:bytes":

v.GCFreedBytes = s.Value.Uint64()

// ... 기타 메트릭 처리

}

}

}
```

프로세스 메트릭 수집

```
func CollectProcessMetrics(refresh time.Duration) {

if !metricsEnabled {

return

}

// 데이터 수집기 초기화
```

```
var (
cpustats = make([]CPUStats, 2)
diskstats = make([]DiskStats, 2)
rstats = make([]runtimeStats, 2)
)
// 메트릭 정의
var (
// CPU 메트릭
cpuSysLoad = GetOrRegisterGauge("system/cpu/sysload", DefaultRegistry)
cpuSysWait = GetOrRegisterGauge("system/cpu/syswait", DefaultRegistry)
cpuProcLoad = GetOrRegisterGauge("system/cpu/procload", DefaultRegistry)
cpuThreads = GetOrRegisterGauge("system/cpu/threads", DefaultRegistry)
cpuGoroutines = GetOrRegisterGauge("system/cpu/goroutines", DefaultRegistry)
// 메모리 메트릭
memPauses = getOrRegisterRuntimeHistogram("system/memory/pauses", secondsToNs, n
memAllocs = GetOrRegisterMeter("system/memory/allocs", DefaultRegistry)
memFrees = GetOrRegisterMeter("system/memory/frees", DefaultRegistry)
memTotal = GetOrRegisterGauge("system/memory/held", DefaultRegistry)
heapUsed = GetOrRegisterGauge("system/memory/used", DefaultRegistry)
heapObjects = GetOrRegisterGauge("system/memory/objects", DefaultRegistry)
// 디스크 메트릭
diskReads = GetOrRegisterMeter("system/disk/readcount", DefaultRegistry)
diskReadBytes = GetOrRegisterMeter("system/disk/readdata", DefaultRegistry)
```

```
diskWrites = GetOrRegisterMeter("system/disk/writecount", DefaultRegistry)
diskWriteBytes = GetOrRegisterMeter("system/disk/writedata", DefaultRegistry)
)
// 주기적 수집 루프
for {
// CPU 통계 수집
ReadCPUStats(&cpustats[1])
if  cpustats[0].GlobalTime != 0 {
// CPU 사용률 계산
sysLoad := cpustats[1].GlobalTime - cpustats[0].GlobalTime
sysWait := cpustats[1].GlobalWait - cpustats[0].GlobalWait
procLoad := cpustats[1].LocalTime - cpustats[0].LocalTime
cpuSysLoad.Update(int64(sysLoad))
cpuSysWait.Update(int64(sysWait))
cpuProcLoad.Update(int64(procLoad))
}
cpustats[0] = cpustats[1]
// 런타임 통계 수집
readRuntimeStats(&rstats[1])
cpuGoroutines.Update(int64(rstats[1].Goroutines))
memTotal.Update(int64(rstats[1].MemTotal))
heapUsed.Update(int64(rstats[1].HeapObjects))
// 디스크 통계 수집
```

```
ReadDiskStats(&diskstats[1])

if  diskstats[0].ReadCount != 0 {

diskReads.Mark(diskstats[1].ReadCount - diskstats[0].ReadCount)

diskReadBytes.Mark(diskstats[1].ReadBytes - diskstats[0].ReadBytes)

diskWrites.Mark(diskstats[1].WriteCount - diskstats[0].WriteCount)

diskWriteBytes.Mark(diskstats[1].WriteBytes - diskstats[0].WriteBytes)

}

diskstats[0] = diskstats[1]

time.Sleep(refresh)

}

}
```

2. 블록체인 특화 메트릭

블록체인 성능 메트릭

```
// core/blockchain.go에서 정의된 메트릭들

var (

// 블록체인 헤드 메트릭

headBlockGauge = metrics.NewRegisteredGauge("chain/head/block", nil)

headHeaderGauge = metrics.NewRegisteredGauge("chain/head/header", nil)

headFastBlockGauge = metrics.NewRegisteredGauge("chain/head/receipt", nil)

headFinalizedBlockGauge = metrics.NewRegisteredGauge("chain/head/finalized", nil

headSafeBlockGauge = metrics.NewRegisteredGauge("chain/head/safe", nil)
```

```
// 체인 정보 및 처리량
chainInfoGauge = metrics.NewRegisteredGaugeInfo("chain/info", nil)
chainMgaspsMeter = metrics.NewRegisteredResettingTimer("chain/mgasps", nil)
// 계정 관련 메트릭
accountReadTimer = metrics.NewRegisteredResettingTimer("chain/account/reads", ni
accountHashTimer = metrics.NewRegisteredResettingTimer("chain/account/hashes", n
accountUpdateTimer = metrics.NewRegisteredResettingTimer("chain/account/updates"
accountCommitTimer = metrics.NewRegisteredResettingTimer("chain/account/commits"
// 스토리지 관련 메트릭
storageReadTimer = metrics.NewRegisteredResettingTimer("chain/storage/reads", ni
storageUpdateTimer = metrics.NewRegisteredResettingTimer("chain/storage/updates"
storageCommitTimer = metrics.NewRegisteredResettingTimer("chain/storage/commits"
// 캐시 히트/미스 메트릭
accountCacheHitMeter = metrics.NewRegisteredMeter("chain/account/reads/cache/pro
accountCacheMissMeter = metrics.NewRegisteredMeter("chain/account/reads/cache/pr
storageCacheHitMeter = metrics.NewRegisteredMeter("chain/storage/reads/cache/pro
storageCacheMissMeter = metrics.NewRegisteredMeter("chain/storage/reads/cache/pr
// 블록 처리 메트릭
blockInsertTimer = metrics.NewRegisteredResettingTimer("chain/inserts", nil)
blockValidationTimer = metrics.NewRegisteredResettingTimer("chain/validation", n
blockExecutionTimer = metrics.NewRegisteredResettingTimer("chain/execution", nil
blockWriteTimer = metrics.NewRegisteredResettingTimer("chain/write", nil)
// 재구성 메트릭
```

```
blockReorgMeter = metrics.NewRegisteredMeter("chain/reorg/executes", nil)
blockReorgAddMeter = metrics.NewRegisteredMeter("chain/reorg/add", nil)
blockReorgDropMeter = metrics.NewRegisteredMeter("chain/reorg/drop", nil)
)
```

트라이 데이터베이스 메트릭

```
// triedb/hashdb/database.go에서 정의된 메트릭들
var (
// 메모리 캐시 메트릭
memcacheCleanHitMeter = metrics.NewRegisteredMeter("hashdb/memcache/clean/hit",
memcacheCleanMissMeter = metrics.NewRegisteredMeter("hashdb/memcache/clean/miss"
memcacheCleanReadMeter = metrics.NewRegisteredMeter("hashdb/memcache/clean/read"
memcacheCleanWriteMeter = metrics.NewRegisteredMeter("hashdb/memcache/clean/writ
memcacheDirtyHitMeter = metrics.NewRegisteredMeter("hashdb/memcache/dirty/hit",
memcacheDirtyMissMeter = metrics.NewRegisteredMeter("hashdb/memcache/dirty/miss"
memcacheDirtyReadMeter = metrics.NewRegisteredMeter("hashdb/memcache/dirty/read"
memcacheDirtyWriteMeter = metrics.NewRegisteredMeter("hashdb/memcache/dirty/writ
// 플러시 메트릭
memcacheFlushTimeTimer = metrics.NewRegisteredResettingTimer("hashdb/memcache/fl
memcacheFlushNodesMeter = metrics.NewRegisteredMeter("hashdb/memcache/flush/node
memcacheFlushBytesMeter = metrics.NewRegisteredMeter("hashdb/memcache/flush/byte
// 가비지 컬렉션 메트릭
```

```
memcacheGCTimeTimer = metrics.NewRegisteredResettingTimer("hashdb/memcache/gc/ti
memcacheGCNodesMeter = metrics.NewRegisteredMeter("hashdb/memcache/gc/nodes", ni
memcacheGCBytesMeter = metrics.NewRegisteredMeter("hashdb/memcache/gc/bytes", ni
// 커밋 메트릭
memcacheCommitTimeTimer = metrics.NewRegisteredResettingTimer("hashdb/memcache/c
memcacheCommitNodesMeter = metrics.NewRegisteredMeter("hashdb/memcache/commit/no
memcacheCommitBytesMeter = metrics.NewRegisteredMeter("hashdb/memcache/commit/by
)
```

3. 설정 매개변수와 튜닝

블록체인 설정 매개변수

```
// BlockChainConfig 구조체의 주요 튜닝 매개변수
type BlockChainConfig struct {
// 트라이 데이터베이스 관련 옵션
TrieCleanLimit  int  // 클린 트라이 노드 캐시 크기 (MB)
TrieDirtyLimit  int  // 더티 트라이 노드 메모리 제한 (MB)
TrieTimeLimit time.Duration // 트라이 플러시 시간 제한
TrieNoAsyncFlush  bool  // 비동기 버퍼 플러시 비활성화 여부
TrieJournalDirectory  string  // 트라이 저널 디렉토리 경로
// 상태 관련 옵션
Preimages  bool  // 트라이 키 프리이미지 저장 여부
```

```
StateHistory uint64 // 유지할 상태 히스토리 블록 수

StateScheme string // 상태 저장 스키마 (hash/path)

ArchiveMode bool // 아카이브 모드 활성화 여부

// 스냅샷 관련 옵션

SnapshotLimit int // 스냅샷 엔트리 캐시 크기 (MB)

SnapshotNoBuild bool // 백그라운드 스냅샷 생성 비활성화

SnapshotWait bool // 시작 시 스냅샷 구성 대기

// 기타 옵션

NoPrefetch bool // 상태 프리페치 비활성화

VmConfig vm.Config // EVM 인터프리터 설정

TxLookupLimit int64 // 트랜잭션 인덱싱 제한

}



// 기본 설정

func DefaultConfig() *BlockChainConfig {

return &BlockChainConfig{

TrieCleanLimit: 256, // 256MB 클린 캐시

TrieDirtyLimit: 256, // 256MB 더티 캐시

TrieTimeLimit: 5 * time.Minute, // 5분 플러시 간격

StateScheme: rawdb.HashScheme, // 해시 스키마 사용

SnapshotLimit: 256, // 256MB 스냅샷 캐시

SnapshotWait: true, // 스냅샷 대기 활성화

TxLookupLimit: -1, // 트랜잭션 인덱싱 비활성화
```

```
}

}
```

노드 설정 매개변수

```
// node/config.go의 주요 설정 매개변수
type Config struct {
// P2P 네트워킹 설정
P2P p2p.Config
// HTTP RPC 설정
HTTPHost  string
HTTPPort  int
HTTPCors []string
HTTPVirtualHosts []string
HTTPModules []string
HTTPTimeouts rpc.HTTPTimeouts
// WebSocket 설정
WSHost  string
WSPort  int
WSOrigins []string
WSModules []string
// 인증 설정
AuthAddr  string
AuthPort  int
```

```
AuthVirtualHosts []string
// 기타 설정
DataDir string
KeyStoreDir string
ExternalSigner string
UseLightweightKDF bool
NoUSB bool
USB bool
}
```

4. 실시간 성능 조정

적응적 캐시 크기 조정

```
type AdaptiveCacheManager struct {
blockchain *BlockChain
config *BlockChainConfig
// 모니터링 메트릭
cacheHitRate float64
memoryUsage float64
diskIOWait float64
// 조정 매개변수
minCacheSize int
maxCacheSize int
```

```
adjustmentRate  float64
// 조정 주기
monitorInterval time.Duration
}

func (acm *AdaptiveCacheManager) Start() {
ticker := time.NewTicker(acm.monitorInterval)
defer  ticker.Stop()
for {
select {
case <-ticker.C:
acm.adjustCacheSize()
}
}
}

func (acm *AdaptiveCacheManager) adjustCacheSize() {
// 캐시 히트율 확인
hitRate := acm.calculateCacheHitRate()
// 메모리 사용률 확인
memUsage := acm.getMemoryUsage()
currentSize := acm.config.TrieCleanLimit
newSize := currentSize
```

```
// 캐시 히트율이 낮고 메모리 여유가 있으면 캐시 증가
if hitRate < 0.8 && memUsage < 0.7 {
newSize = int(float64(currentSize) * (1 + acm.adjustmentRate))
if newSize > acm.maxCacheSize {
newSize = acm.maxCacheSize
}
}
// 메모리 사용률이 높으면 캐시 감소
if memUsage > 0.85 {
newSize = int(float64(currentSize) * (1 - acm.adjustmentRate))
if newSize < acm.minCacheSize {
newSize = acm.minCacheSize
}
}
if newSize != currentSize {
log.Info("Adjusting cache size",
"old", currentSize, "new", newSize,
"hitRate", hitRate, "memUsage", memUsage)
acm.config.TrieCleanLimit = newSize
acm.blockchain.ReconfigureCache(acm.config)
}
}
```

```
func (acm *AdaptiveCacheManager) calculateCacheHitRate() float64 {
hits := accountCacheHitMeter.Snapshot().Count() + storageCacheHitMeter.Snapshot(
misses := accountCacheMissMeter.Snapshot().Count() + storageCacheMissMeter.Snaps
total := hits + misses
if total == 0 {
return 1.0
}
return float64(hits) / float64(total)
}
```

자동 튜닝 시스템

```
type AutoTuner struct {
blockchain *BlockChain
config *BlockChainConfig
// 성능 기준선
baselineMetrics map[string]float64
// 튜닝 히스토리
tuningHistory []TuningAction
// 성능 목표
targetTPS float64
targetLatency time.Duration
maxMemoryUsage float64
```

```
}

type TuningAction struct {
Timestamp time.Time
Parameter  string
OldValue  interface{}
NewValue  interface{}
Performance  float64
}

func (at *AutoTuner) OptimizePerformance() {
// 현재 성능 측정
currentTPS := at.measureTPS()
currentLatency := at.measureLatency()
currentMemUsage := at.measureMemoryUsage()
// 성능 목표와 비교
if  currentTPS < at.targetTPS {
at.optimizeForThroughput()
}
if  currentLatency > at.targetLatency {
at.optimizeForLatency()
}
if  currentMemUsage > at.maxMemoryUsage {
```

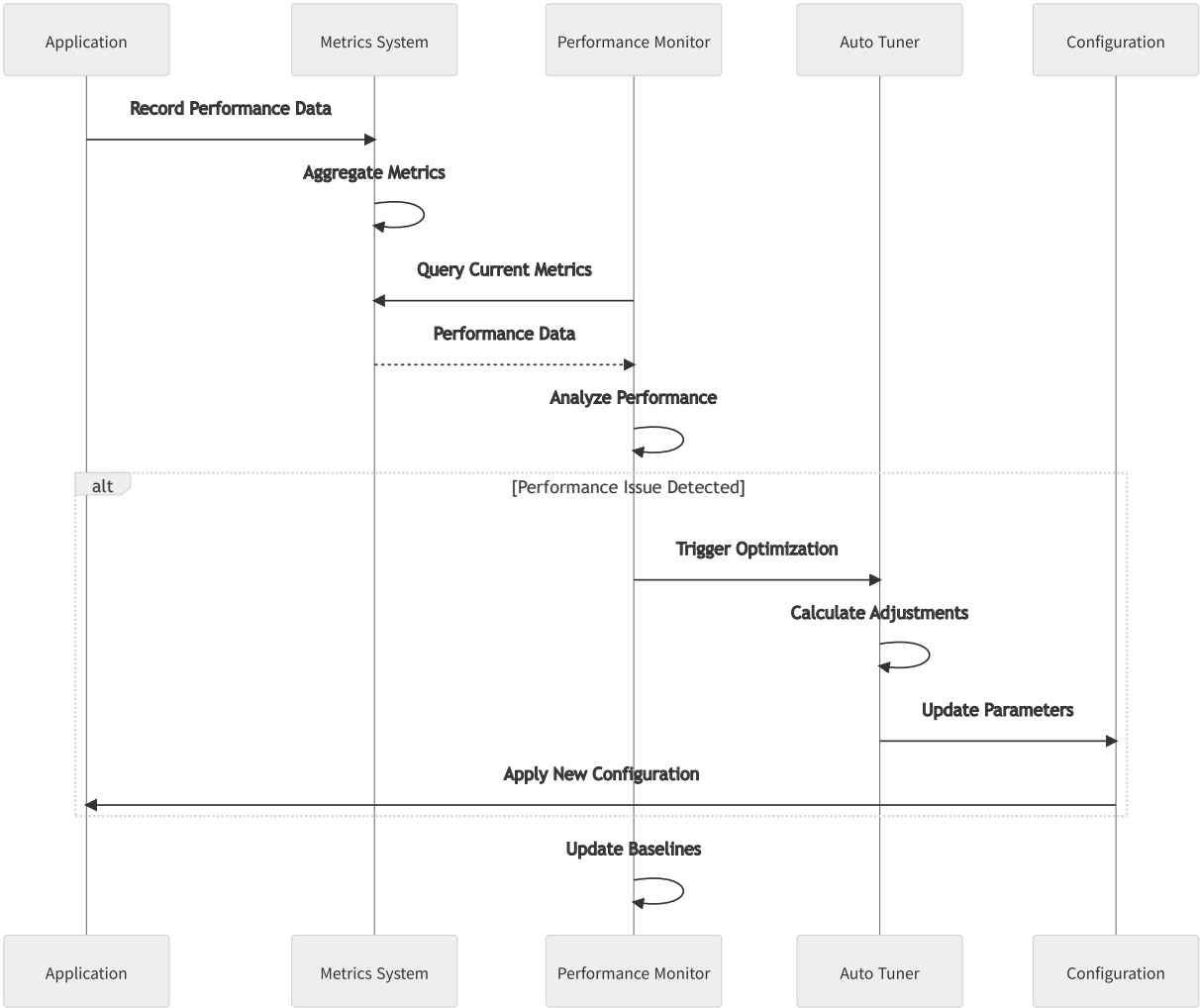
```
at.optimizeForMemory()
}
}

func (at *AutoTuner) optimizeForThroughput() {
// 처리량 최적화를 위한 매개변수 조정
// 1. 캐시 크기 증가
if  at.config.TrieCleanLimit < 512 {
oldValue := at.config.TrieCleanLimit
at.config.TrieCleanLimit = min(oldValue*2, 512)
at.recordTuningAction("TrieCleanLimit", oldValue, at.config.TrieCleanLimit)
}
// 2. 비동기 플러시 활성화
if  at.config.TrieNoAsyncFlush {
at.config.TrieNoAsyncFlush = false
at.recordTuningAction("TrieNoAsyncFlush", true, false)
}
// 3. 프리페치 활성화
if  at.config.NoPrefetch {
at.config.NoPrefetch = false
at.recordTuningAction("NoPrefetch", true, false)
}
}
```

```go
func (at *AutoTuner) optimizeForLatency() {
// 지연시간 최적화를 위한 매개변수 조정
// 1. 플러시 간격 단축
if  at.config.TrieTimeLimit > time.Minute {
oldValue := at.config.TrieTimeLimit
at.config.TrieTimeLimit = max(oldValue/2, time.Minute)
at.recordTuningAction("TrieTimeLimit", oldValue, at.config.TrieTimeLimit)
}
// 2. 스냅샷 대기 비활성화
if  at.config.SnapshotWait {
at.config.SnapshotWait = false
at.recordTuningAction("SnapshotWait", true, false)
}
}

func (at *AutoTuner) optimizeForMemory() {
// 메모리 사용량 최적화를 위한 매개변수 조정
// 1. 캐시 크기 감소
if  at.config.TrieCleanLimit > 128 {
oldValue := at.config.TrieCleanLimit
at.config.TrieCleanLimit = max(oldValue/2, 128)
at.recordTuningAction("TrieCleanLimit", oldValue, at.config.TrieCleanLimit)
```

```
}

// 2. 스냅샷 캐시 크기 감소

if  at.config.SnapshotLimit > 128 {

oldValue := at.config.SnapshotLimit

at.config.SnapshotLimit = max(oldValue/2, 128)

at.recordTuningAction("SnapshotLimit", oldValue, at.config.SnapshotLimit)

}

}
```

실행 흐름

성능 모니터링 파이프라인


Application
Metrics System
Performance Monitor
Auto Tuner
Configuration
Record Performance Data
Aggregate Metrics
Query Current Metrics
Performance Data
Analyze Performance
alt
[Performance Issue Detected]
Trigger Optimization
Calculate Adjustments
Update Parameters
Apply New Configuration
Update Baselines
Application
Metrics System
Performance Monitor
Auto Tuner
Configuration

병목 지점 식별 프로세스

```go
type BottleneckDetector struct {
metrics  map[string][]float64
thresholds  map[string]float64
// 분석 윈도우
windowSize  int
// 병목 유형
bottleneckTypes  map[string]BottleneckType
}

type BottleneckType int

const (
CPUBottleneck BottleneckType = iota
MemoryBottleneck
IOBottleneck
NetworkBottleneck
CacheBottleneck
)

func (bd *BottleneckDetector) DetectBottlenecks() []Bottleneck {
var  bottlenecks []Bottleneck
// CPU 병목 검사
```

```
if bd.isCPUBottleneck() {
bottlenecks = append(bottlenecks, Bottleneck{
Type: CPUBottleneck,
Severity: bd.calculateSeverity("cpu"),
Recommendations: []string{
"Reduce concurrent operations",
"Optimize CPU-intensive algorithms",
"Consider horizontal scaling",
},
})
}
// 메모리 병목 검사
if bd.isMemoryBottleneck() {
bottlenecks = append(bottlenecks, Bottleneck{
Type: MemoryBottleneck,
Severity: bd.calculateSeverity("memory"),
Recommendations: []string{
"Reduce cache sizes",
"Enable garbage collection tuning",
"Implement memory pooling",
},
})
}
```

```
// I/O 병목 검사
if  bd.isIOBottleneck() {
bottlenecks = append(bottlenecks, Bottleneck{
Type: IOBottleneck,
Severity: bd.calculateSeverity("io"),
Recommendations: []string{
"Increase batch sizes",
"Use SSD storage",
"Optimize disk access patterns",
},
})
}
return  bottlenecks
}


func (bd *BottleneckDetector) isCPUBottleneck() bool {
cpuUsage := bd.getAverageMetric("system/cpu/procload")
return  cpuUsage > bd.thresholds["cpu_high"]
}


func (bd *BottleneckDetector) isMemoryBottleneck() bool {
memUsage := bd.getAverageMetric("system/memory/used")
memTotal := bd.getAverageMetric("system/memory/held")
```

```
if  memTotal == 0 {
return false
}
usageRatio := memUsage / memTotal
return  usageRatio > bd.thresholds["memory_high"]
}

func (bd *BottleneckDetector) isIOBottleneck() bool {
ioWait := bd.getAverageMetric("system/cpu/syswait")
return  ioWait > bd.thresholds["io_wait_high"]
}
```

실제 사용 예시

1. 메트릭 기반 알림 시스템

```
type AlertManager struct {
registry Registry
rules []AlertRule
// 알림 채널
emailNotifier EmailNotifier
slackNotifier SlackNotifier
webhookNotifier WebhookNotifier
```

```
}

type AlertRule struct {
Name string
MetricName string
Condition string // "gt", "lt", "eq"
Threshold float64
Duration time.Duration
Severity AlertSeverity
Description string
}

type AlertSeverity int

const (
Info AlertSeverity = iota
Warning
Critical
)

func (am *AlertManager) CheckAlerts() {
for _, rule := range am.rules {
metric := am.registry.Get(rule.MetricName)
```

```
if metric == nil {
continue
}
currentValue := am.extractValue(metric)
if am.evaluateCondition(currentValue, rule.Condition, rule.Threshold) {
alert := Alert{
Rule: rule,
Value: currentValue,
Timestamp: time.Now(),
}
am.sendAlert(alert)
}
}
}


func (am *AlertManager) sendAlert(alert Alert) {
switch alert.Rule.Severity {
case Critical:
am.emailNotifier.Send(alert)
am.slackNotifier.Send(alert)
am.webhookNotifier.Send(alert)
case Warning:
am.slackNotifier.Send(alert)
```

```
case  Info:
// 로그만 기록
log.Info("Alert triggered", "rule", alert.Rule.Name, "value", alert.Value)
}
}
```

2. 성능 대시보드 데이터 제공

```
type DashboardAPI struct {
registry Registry
history *MetricHistory
}

type MetricHistory struct {
data  map[string][]TimestampedValue
mutex sync.RWMutex
maxAge time.Duration
}

type TimestampedValue struct {
Timestamp time.Time
Value  float64
}
```

```go
func (api *DashboardAPI) GetMetrics(w http.ResponseWriter, r *http.Request) {
metrics := make(map[string]interface{})
// 현재 메트릭 수집
api.registry.Each(func(name  string, metric  interface{}) {
switch  m := metric.(type) {
case *Counter:
metrics[name] = map[string]interface{}{
"type": "counter",
"value": m.Snapshot().Count(),
}
case *Gauge:
metrics[name] = map[string]interface{}{
"type": "gauge",
"value": m.Snapshot().Value(),
}
case *Timer:
snapshot := m.Snapshot()
metrics[name] = map[string]interface{}{
"type": "timer",
"count": snapshot.Count(),
"mean": snapshot.Mean(),
"p95": snapshot.Percentile(0.95),
```

```
"p99": snapshot.Percentile(0.99),

}

}

})

// JSON 응답

w.Header().Set("Content-Type", "application/json")

json.NewEncoder(w).Encode(metrics)

}



func (api *DashboardAPI) GetHistoricalData(w http.ResponseWriter, r *http.Reques

metricName := r.URL.Query().Get("metric")

duration := r.URL.Query().Get("duration")

d, err := time.ParseDuration(duration)

if  err != nil {

d = time.Hour  // 기본값: 1시간

}

data := api.history.GetData(metricName, d)

w.Header().Set("Content-Type", "application/json")

json.NewEncoder(w).Encode(data)

}
```

3. 하드웨어별 최적화 가이드

```
type HardwareOptimizer struct {
cpuCores int
totalMemory uint64
diskType string // "HDD", "SSD", "NVMe"
networkBW uint64 // Mbps
}

func (ho *HardwareOptimizer) GenerateOptimalConfig() *BlockChainConfig {
config := DefaultConfig()
// CPU 기반 최적화
if ho.cpuCores >= 8 {
// 고성능 CPU: 더 많은 동시 작업 허용
config.NoPrefetch = false
config.TrieNoAsyncFlush = false
} else if ho.cpuCores <= 2 {
// 저성능 CPU: 동시 작업 제한
config.NoPrefetch = true
config.TrieTimeLimit = 10 * time.Minute
}
// 메모리 기반 최적화
memoryGB := ho.totalMemory / (1024 * 1024 * 1024)
if memoryGB >= 32 {
```

```
// 대용량 메모리: 큰 캐시 사용
config.TrieCleanLimit = 1024  // 1GB
config.TrieDirtyLimit = 1024  // 1GB
config.SnapshotLimit = 512  // 512MB
} else  if  memoryGB >= 16 {
// 중간 메모리: 기본 설정
config.TrieCleanLimit = 512  // 512MB
config.TrieDirtyLimit = 512  // 512MB
config.SnapshotLimit = 256  // 256MB
} else {
// 저용량 메모리: 작은 캐시 사용
config.TrieCleanLimit = 128  // 128MB
config.TrieDirtyLimit = 128  // 128MB
config.SnapshotLimit = 64  // 64MB
}
// 디스크 타입 기반 최적화
switch  ho.diskType {
case  "NVMe":
// 고속 스토리지: 빈번한 플러시 허용
config.TrieTimeLimit = time.Minute
config.SnapshotNoBuild = false
case  "SSD":
// SSD: 균형잡힌 설정
```

```
config.TrieTimeLimit = 3 * time.Minute

config.SnapshotNoBuild = false

case "HDD":

// HDD: 플러시 빈도 감소

config.TrieTimeLimit = 10 * time.Minute

config.SnapshotNoBuild = true

}

return config

}



func (ho *HardwareOptimizer) GetRecommendations() []string {

var recommendations []string

if ho.cpuCores < 4 {

recommendations = append(recommendations,

"Consider upgrading to at least 4 CPU cores for better performance")

}

memoryGB := ho.totalMemory / (1024 * 1024 * 1024)

if memoryGB < 8 {

recommendations = append(recommendations,

"Minimum 8GB RAM recommended, 16GB+ preferred")

}

if ho.diskType == "HDD" {

recommendations = append(recommendations,
```

```
"SSD storage highly recommended for blockchain data")

}

if  ho.networkBW < 100 {

recommendations = append(recommendations,

"Fast internet connection (100+ Mbps) recommended for sync performance")

}

return  recommendations

}
```

관련 시스템

의존성 관계

1. **메트릭 수집 시스템**: 모든 성능 데이터의 중앙 집중화
2. **설정 관리 시스템**: 동적 매개변수 조정
3. **알림 시스템**: 성능 이슈 실시간 감지
4. **로깅 시스템**: 상세한 성능 분석을 위한 로그 수집

상호작용 패턴

- **반응형 튜닝**: 성능 메트릭 기반 실시간 조정
- **예측적 최적화**: 과거 패턴 분석을 통한 사전 조정
- **적응적 학습**: 시스템 동작 패턴 학습을 통한 자동 최적화

영향 범위

성능 모니터링과 튜닝은 다음 영역에 영향을 미칩니다:

- **시스템 안정성**: 성능 이슈 조기 감지로 안정성 향상
- **사용자 경험**: 최적화된 응답 시간과 처리량 제공
- **운영 효율성**: 자동화된 튜닝으로 관리 부담 감소
- **리소스 활용**: 하드웨어 리소스의 최적 활용

이러한 포괄적인 성능 모니터링과 튜닝 시스템을 통해 Geth는 다양한 환경에서 최적의 성능을 유지하며, 운영자에게 시스템 상태에 대한 깊은 통찰력을 제공합니다.